

Scapegoat Trees

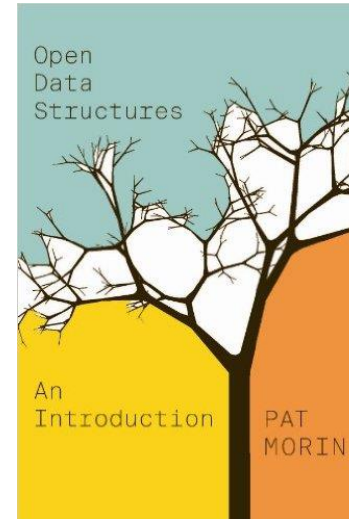
COMP 2402

Abstract Data Types & Algorithms

Readings

Today's topic

- Scapegoat Trees
– Chapter 8



Sorted Sets

Skip Lists

find, add, remove

expected $O(\log n)$

BST

$O(\text{height}) \in O(n)$

Random BST

expected $O(\log n)$ [find]

Treap

expected $O(\log n)$

Scapegoat Trees

A **scapegoat tree** is a binary search tree that keeps itself balanced by partially rebuilding parts of itself when necessary.

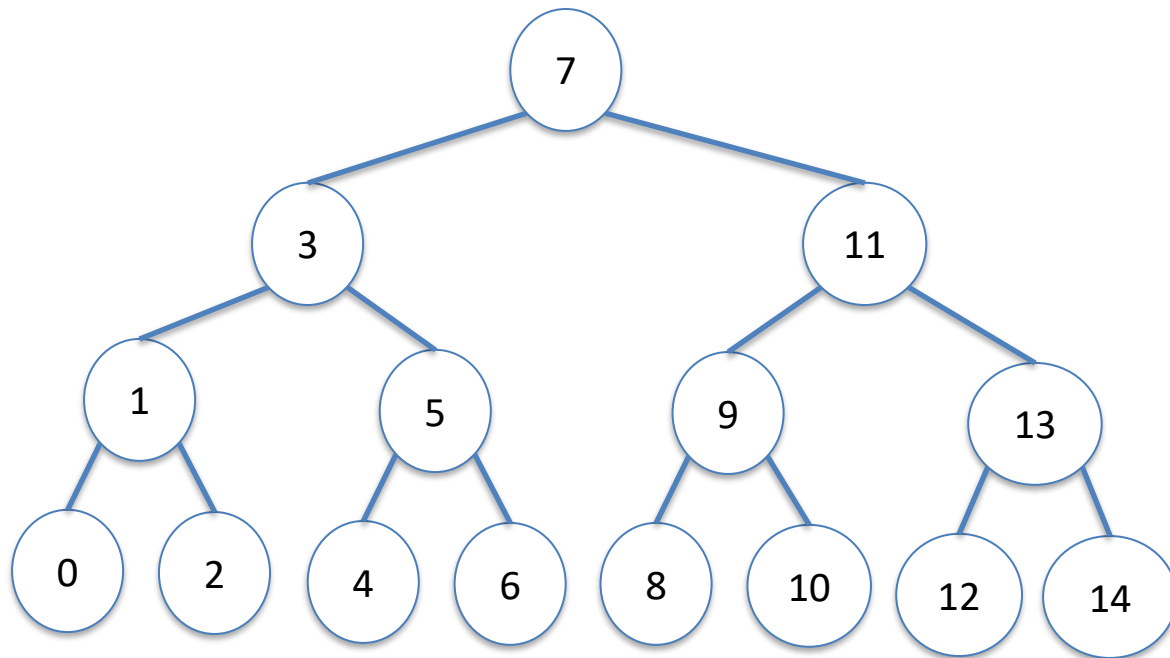
It is a *lazy* data structure. It only does extra work when it needs to.

Find has $O(\log n)$ cost in the **worst case**.

Add/remove have amortized $O(\log n)$ cost.

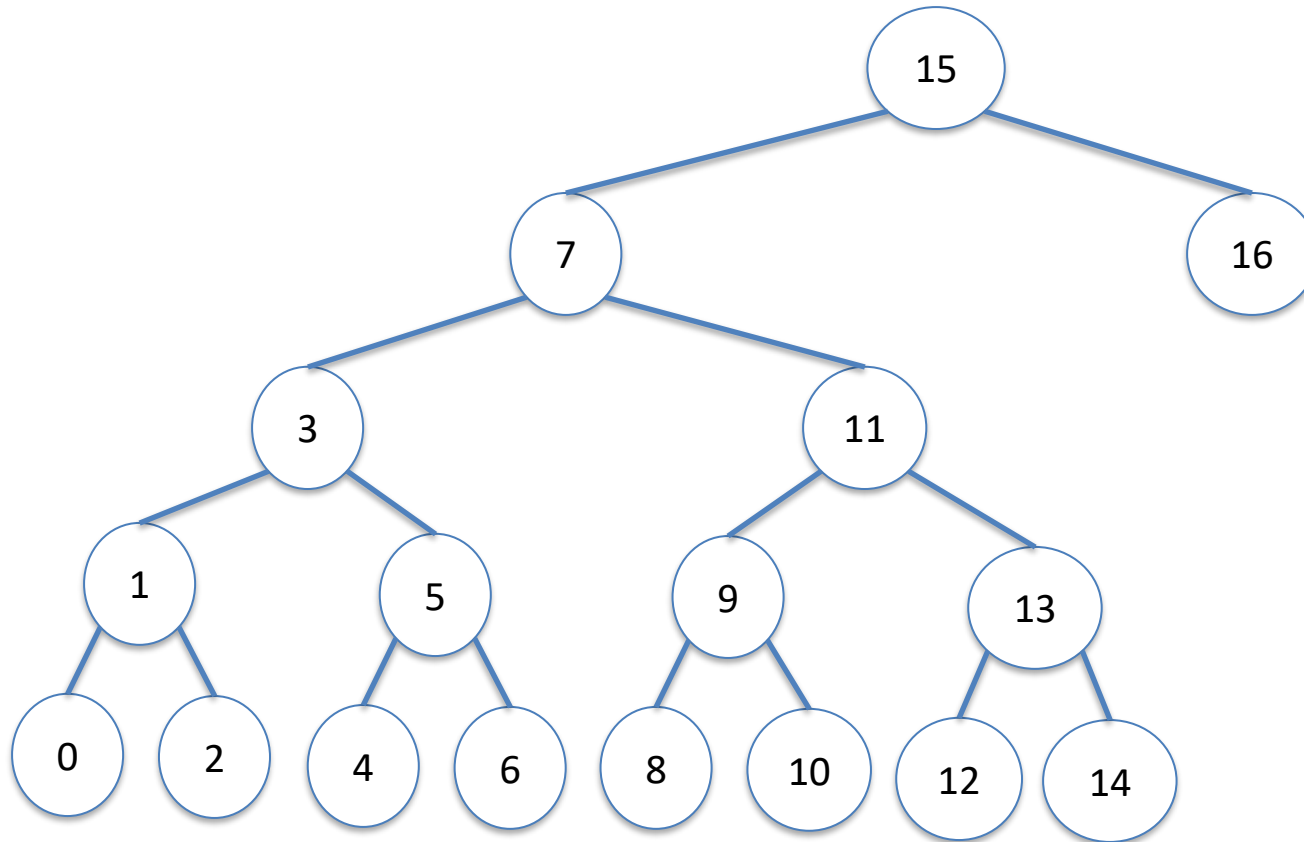
Scapegoat Trees

What can we say about the following BST?



Scapegoat Trees

What can we say about the following BST?



Scapegoat Trees

Expected height of a random binary search tree is

$$O(\log n)$$

Why?

Scapegoat Trees

A **scapegoat tree** keeps track of two parameters

- n is the number of nodes in the tree
- q is an overestimate of n

The tree always maintains

- $\frac{1}{2}q \leq n \leq q \leq 2n$
- $depth(u) \leq \log_{3/2}(q)$ for all nodes u

Scapegoat Trees

A **scapegoat tree** will **rebuild** itself or parts of itself when needed.

Rebuilding a subtree results in a new subtree what has minimal height.

We can rebuild a subtree that has n nodes in time $O(n)$.

Scapegoat Trees

Removing a value from a **scapegoat tree**

- Remove as normal for a BST
- Decrement n
- Check if $n < \frac{1}{2}q$
 - If so then rebuild entire tree and set $q = n$

Scapegoat Trees

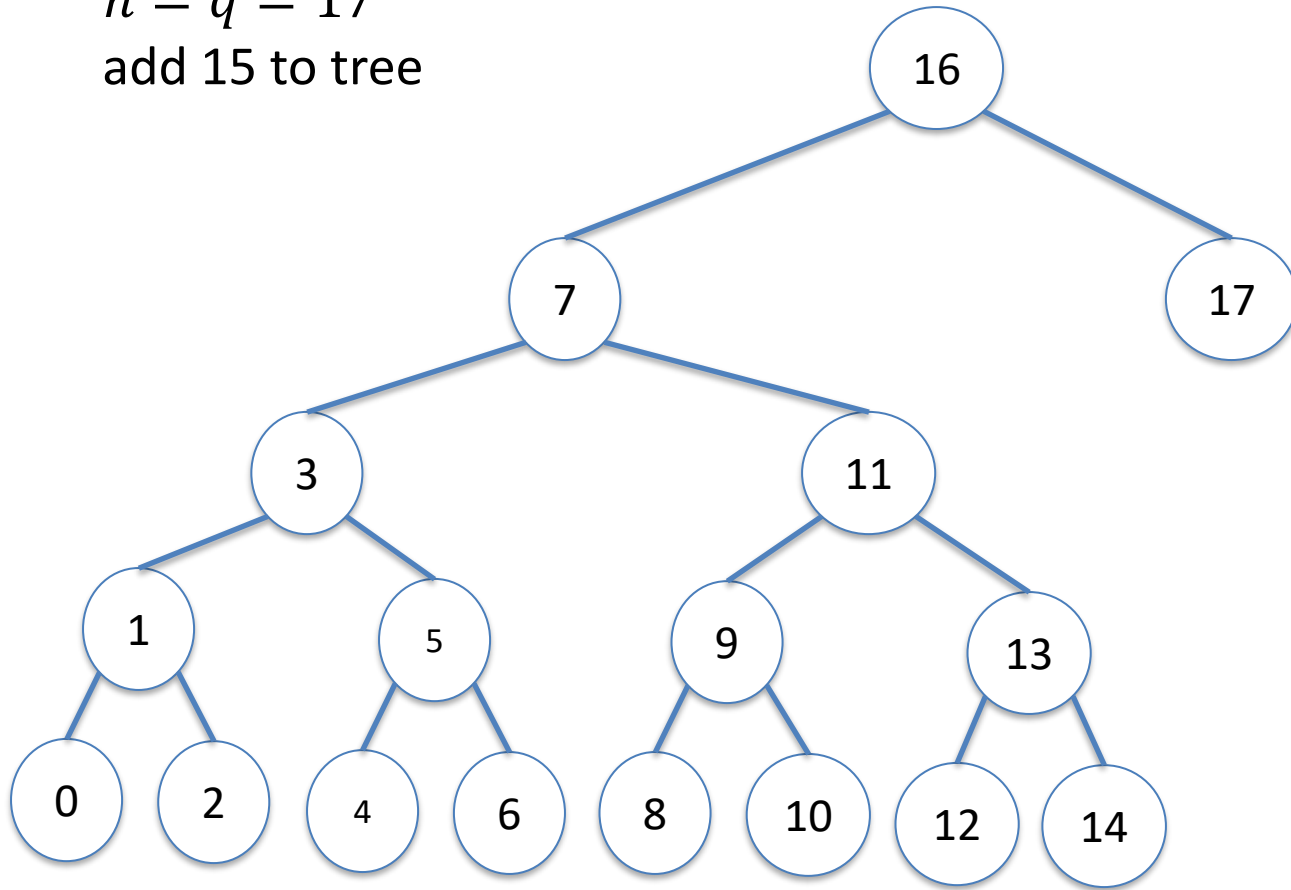
Adding a value to a **scapegoat tree**

- Add value as normal for a BST (insert node u)
- Increment both n and q
- Check if $depth(u) > \log_{3/2} q$
 - If so then find a scapegoat w on path from u to root

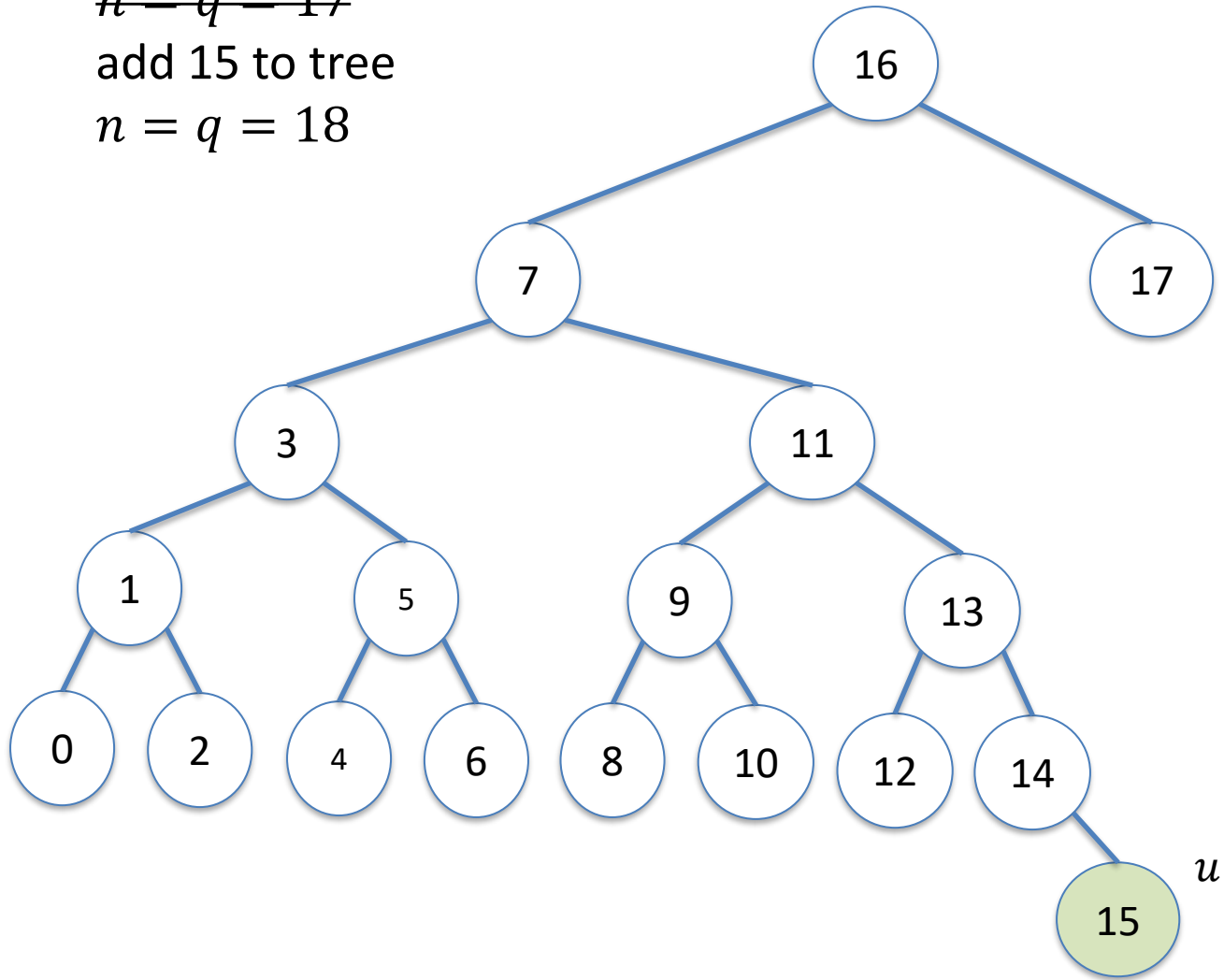
$$size(w) < \frac{3}{2} size(w.child)$$

Rebuild the subtree rooted at w

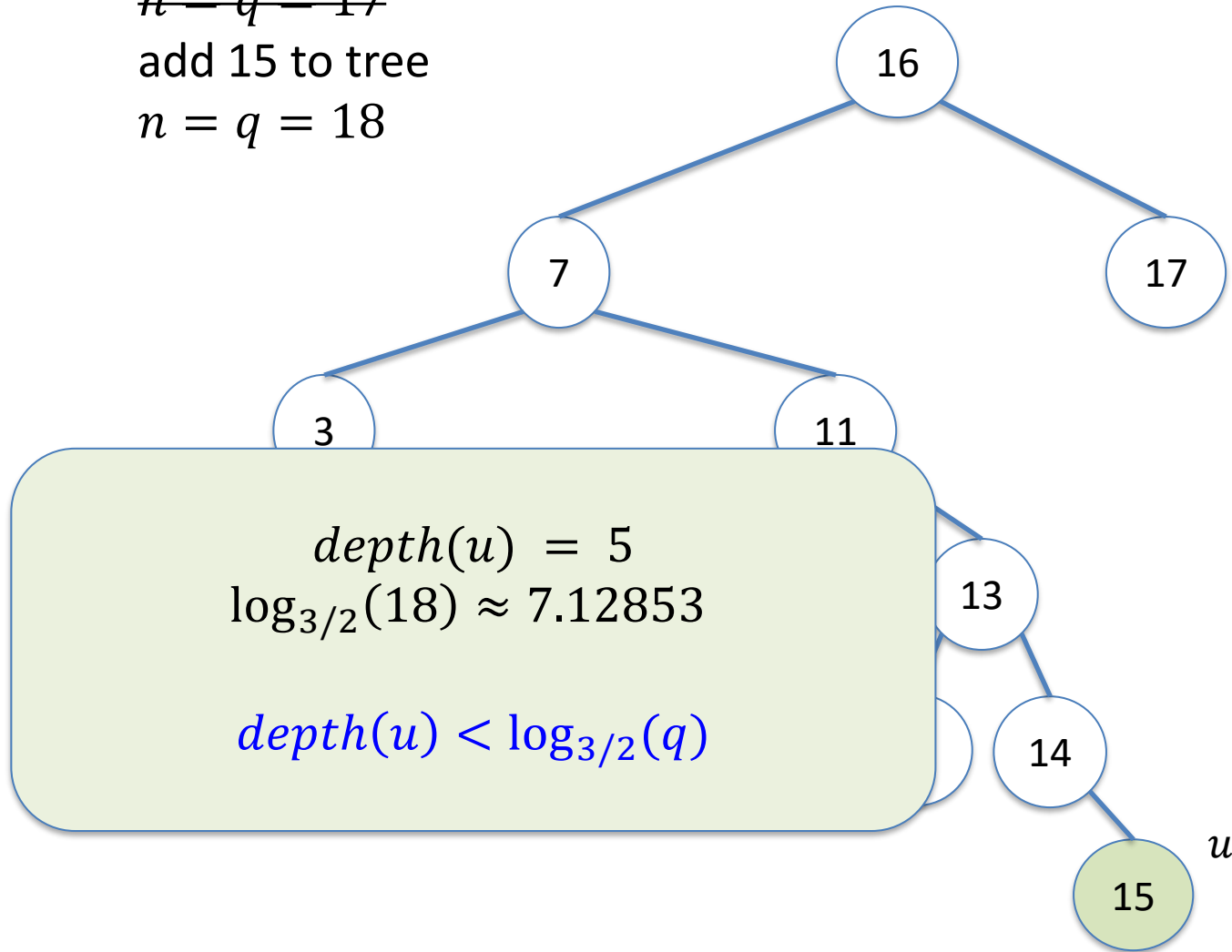
$n = q = 17$
add 15 to tree



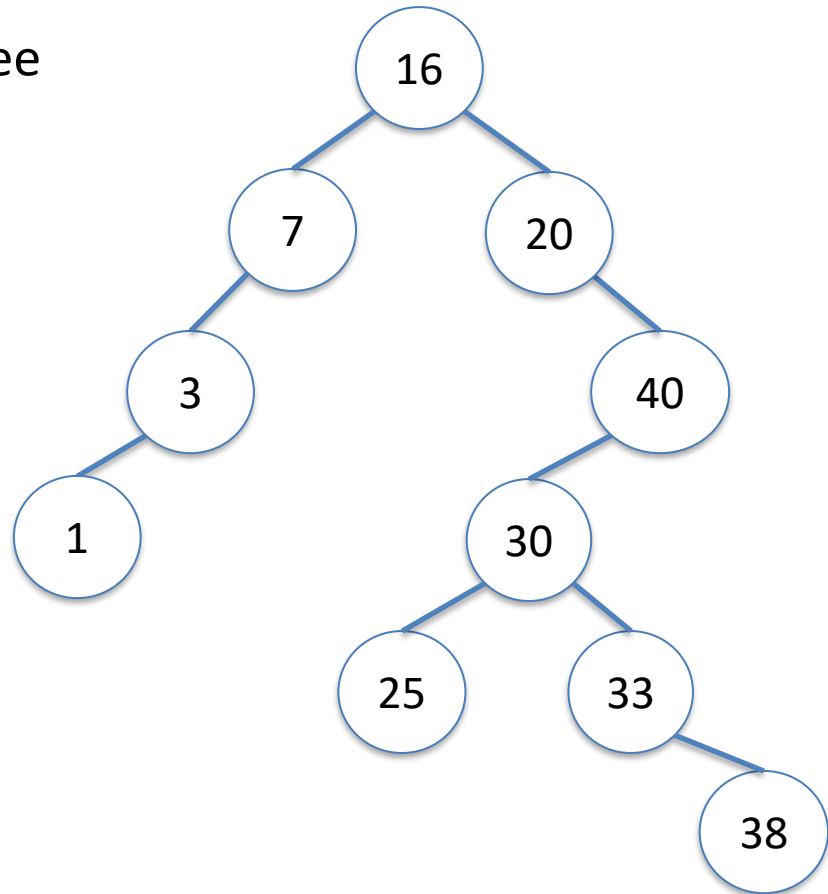
~~$n = q = 17$~~
add 15 to tree
 $n = q = 18$



~~$n = q = 17$~~
add 15 to tree
 $n = q = 18$



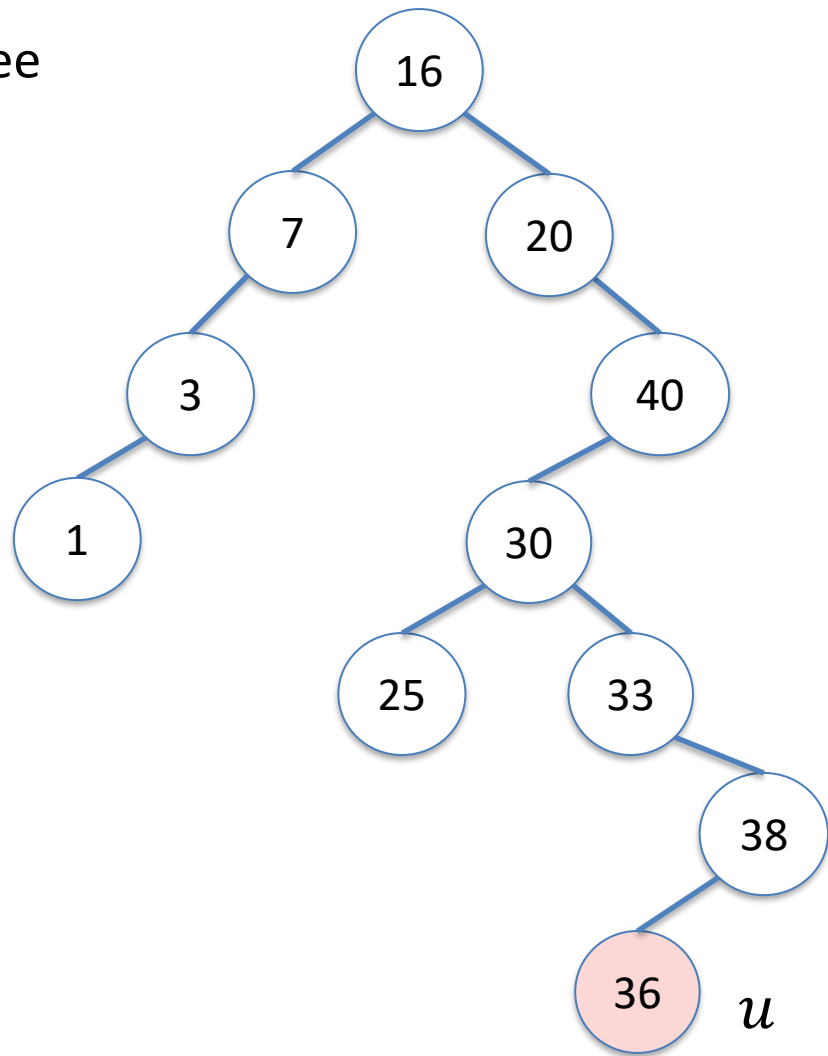
$n = q = 10$
add 36 to tree



~~$n = q = 10$~~

add 36 to tree

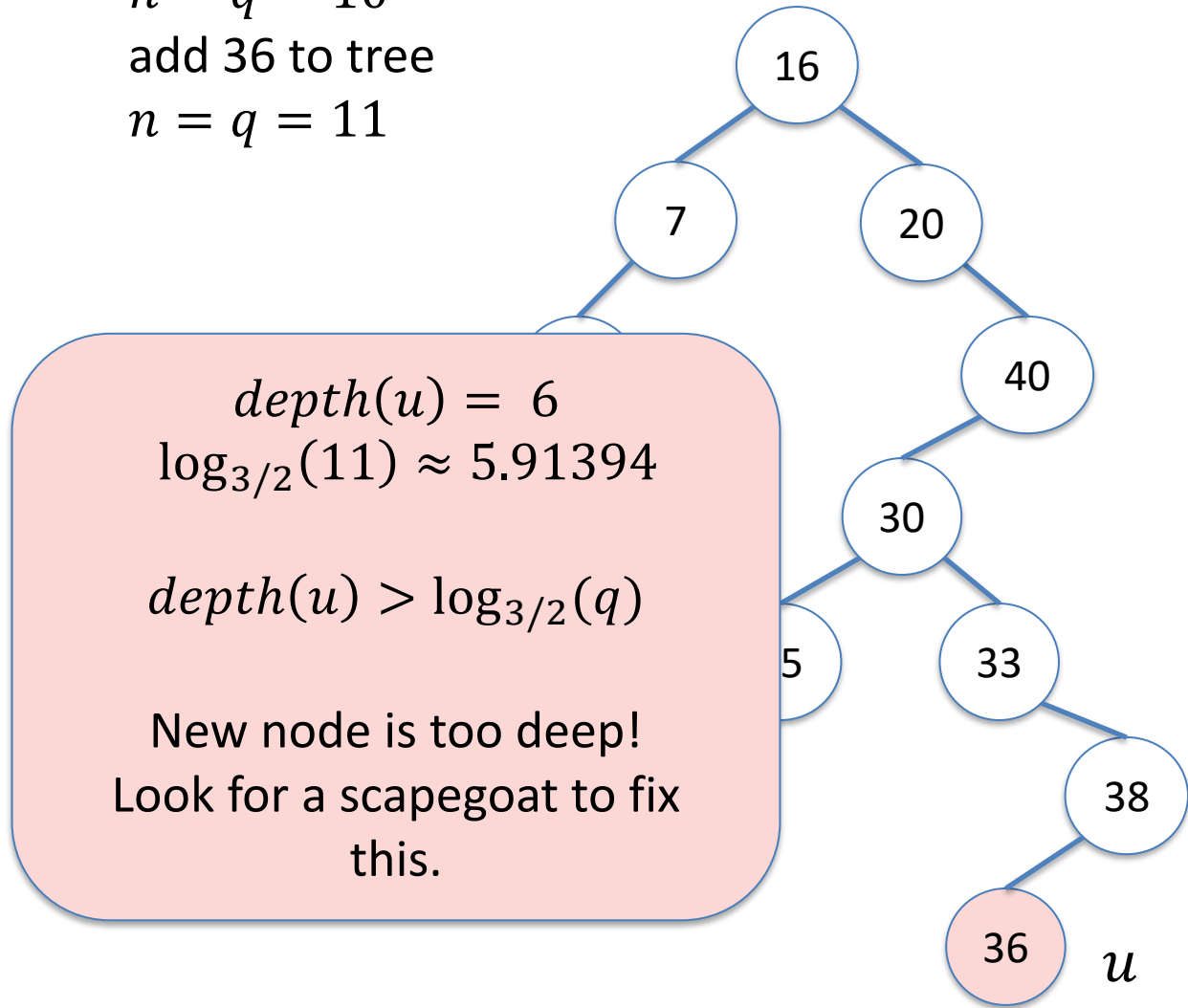
$n = q = 11$



~~$n = q = 10$~~

add 36 to tree

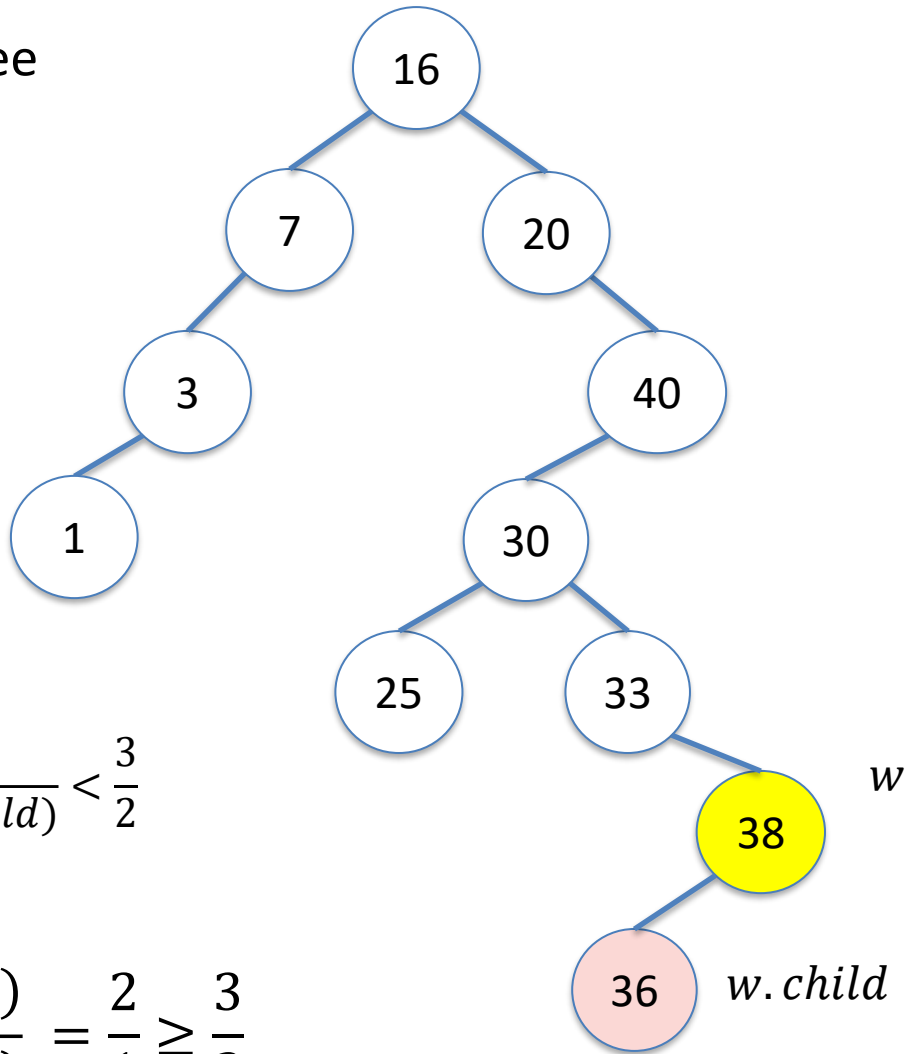
$n = q = 11$



~~$n = q = 10$~~

add 36 to tree

$n = q = 11$



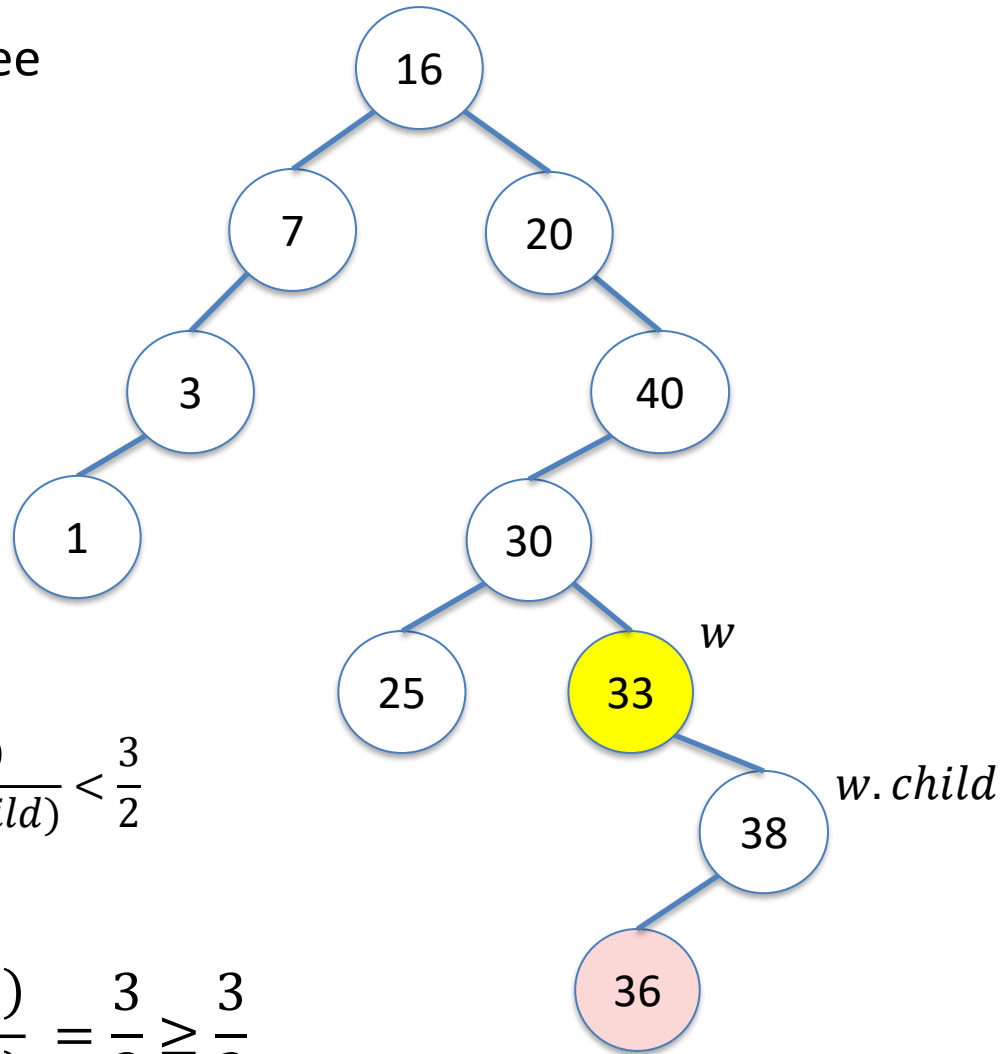
check if $\frac{size(w)}{size(w.child)} < \frac{3}{2}$

$$\frac{size(38)}{size(36)} = \frac{2}{1} \geq \frac{3}{2}$$

~~$n = q = 10$~~

add 36 to tree

$n = q = 11$



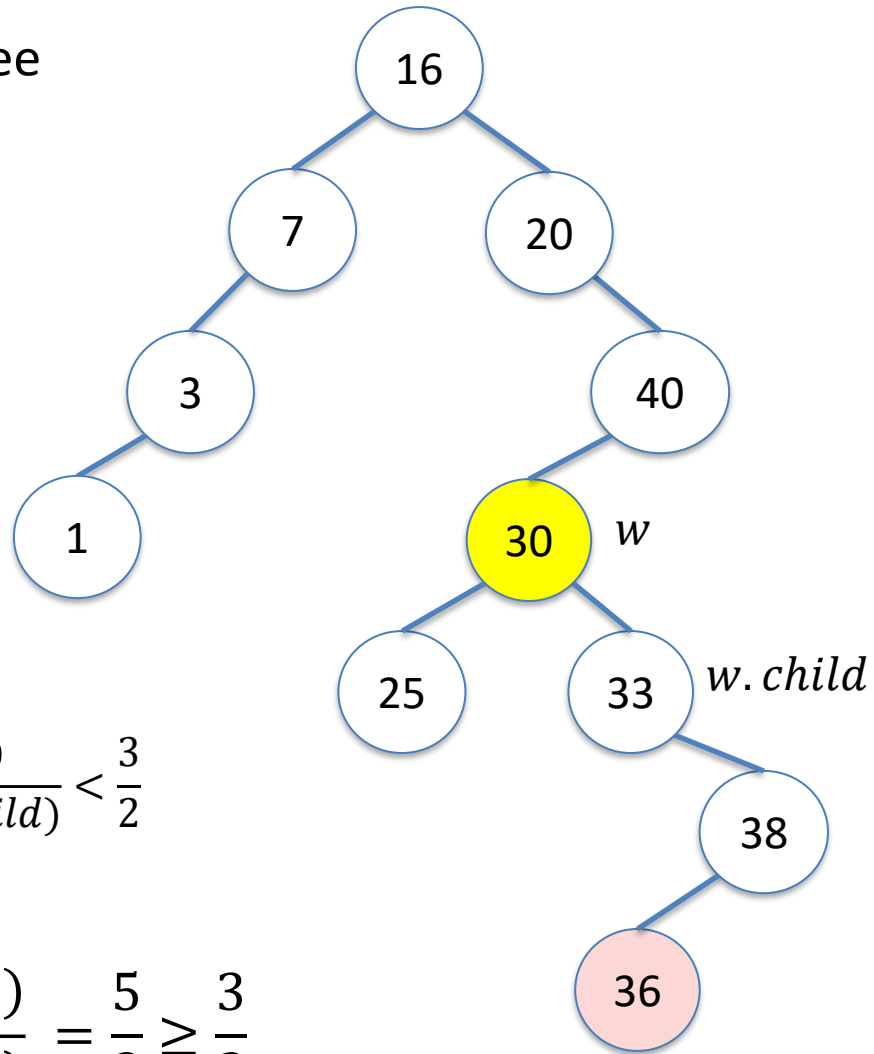
check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(33)}{\text{size}(38)} = \frac{3}{2} \geq \frac{3}{2}$$

~~$n = q = 10$~~

add 36 to tree

$n = q = 11$



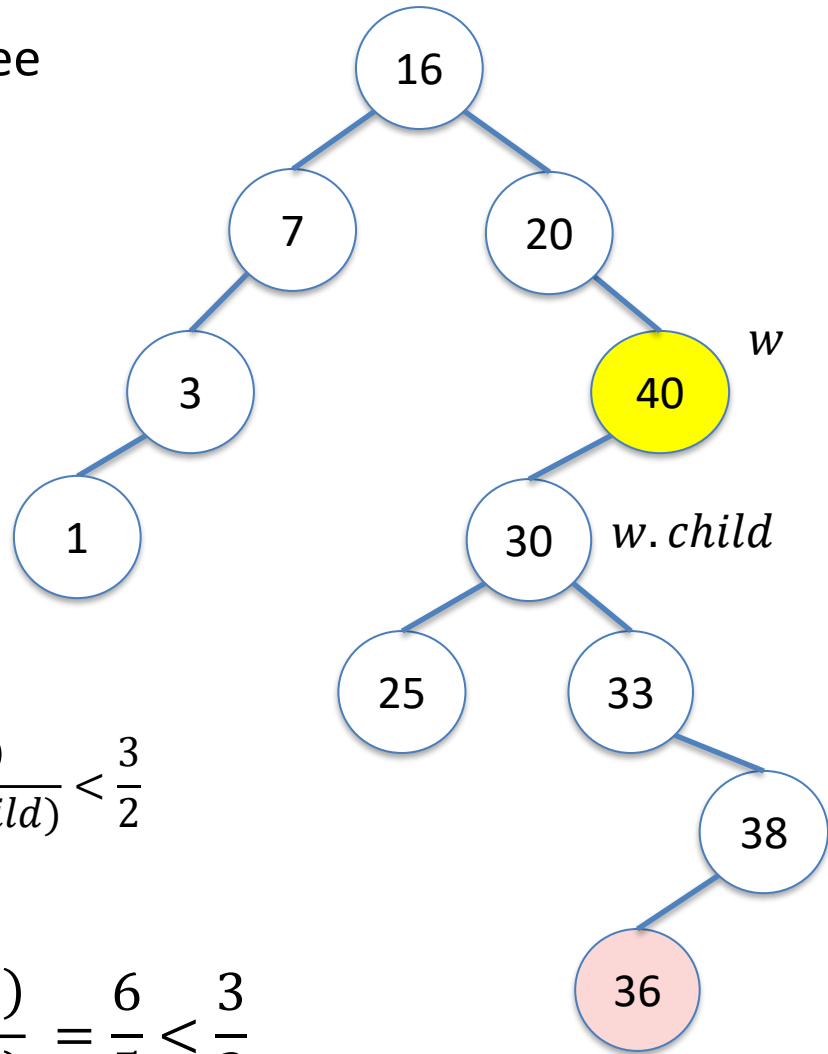
check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(30)}{\text{size}(33)} = \frac{5}{3} \geq \frac{3}{2}$$

~~$n = q = 10$~~

add 36 to tree

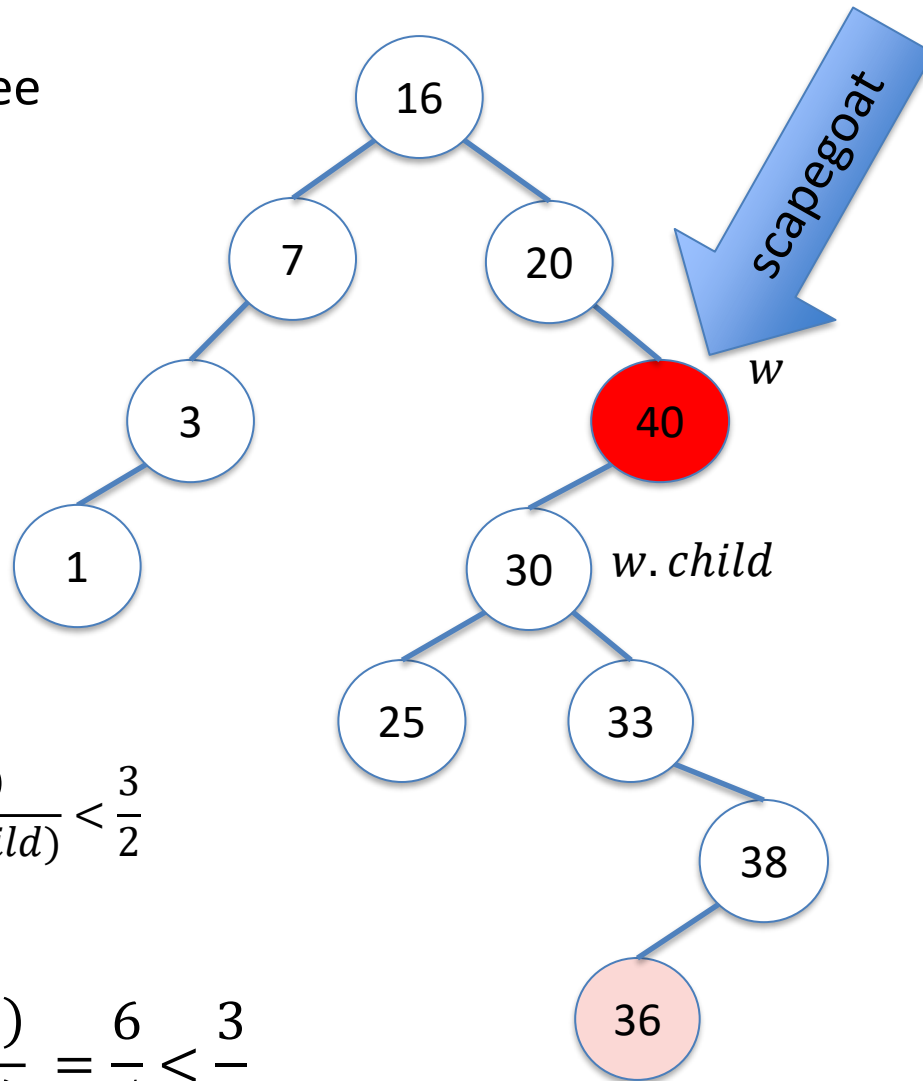
$n = q = 11$



check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(40)}{\text{size}(30)} = \frac{6}{5} < \frac{3}{2}$$

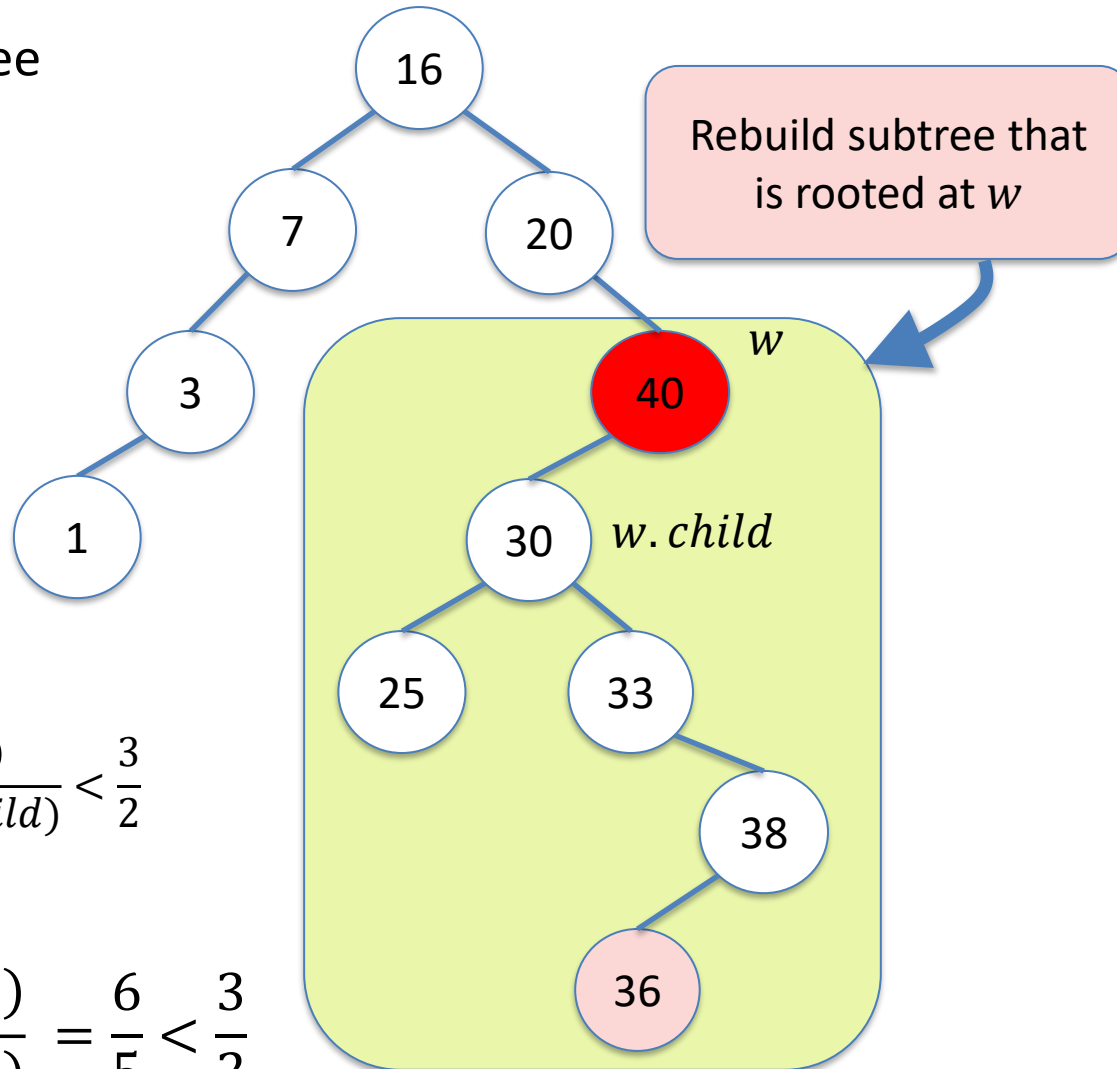
~~$n = q = 10$~~
 add 36 to tree
 $n = q = 11$



check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(40)}{\text{size}(30)} = \frac{6}{5} < \frac{3}{2}$$

~~$n = q = 10$~~
 add 36 to tree
 $n = q = 11$



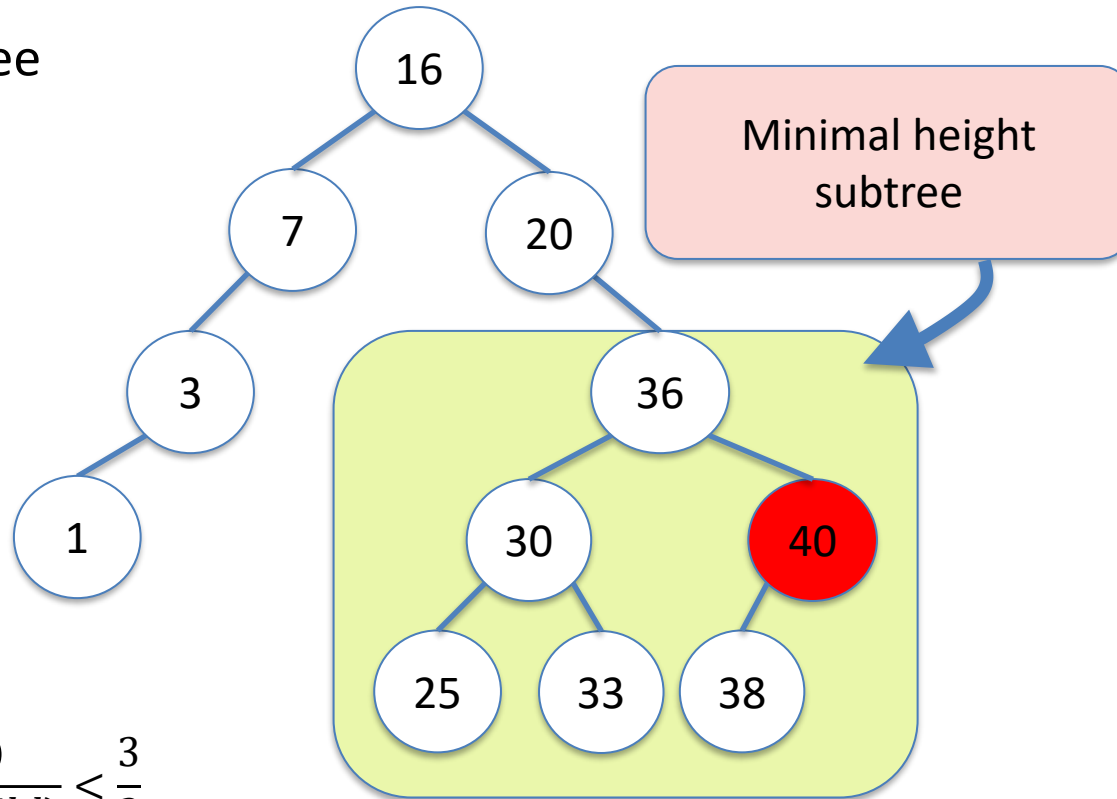
check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(40)}{\text{size}(30)} = \frac{6}{5} < \frac{3}{2}$$

~~$n = q = 10$~~

add 36 to tree

$n = q = 11$



check if $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

$$\frac{\text{size}(40)}{\text{size}(30)} = \frac{6}{5} < \frac{3}{2}$$

Scapegoat Trees

We claimed earlier that

- Find has $O(\log n)$ cost in the worst case.
- Add/remove have amortized $O(\log n)$ cost.

How can we show this?

What do we need to show to justify this?

Scapegoat Trees

We need to know the cost of rebuild.

The **rebuild** operation takes a binary tree and completely reconstructs it so that it is as balanced as possible.

Rebuilding the subtree rooted at node u costs

$$O(\text{size}(u)) \in O(n)$$

Scapegoat Trees

Lemma 8.1 There is always a scapegoat when adding a node u that has $\text{depth}(u) > \log_{3/2} q$

Recall: Scapegoat found when $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

Scapegoat Trees

Comment Rebuilding the scapegoat causes the height of the subtree rooted at the scapegoat to decrease by at least 1.

Recall: Scapegoat found when $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

Scapegoat Trees

Lemma 8.2 Finding the scapegoat w and rebuilding costs $O(\text{size}(w))$.

Recall: Scapegoat found when $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

Scapegoat Trees

Lemma 8.2 Finding the scapegoat w and rebuilding costs $O(\text{size}(w))$.

Recall: Scapegoat found when $\frac{\text{size}(w)}{\text{size}(w.\text{child})} < \frac{3}{2}$

Scapegoat Trees

Lemma 8.3 Starting with an empty scapegoat tree any sequence of m add/remove operations has cost $O(m \log m)$ for all rebuild operations.

Scapegoat Trees

Theorem 8.1 A **ScapegoatTree** implements the SSet interface. Ignoring the cost of rebuild operations, a ScapegoatTree supports the operations `add()`, `remove()` and `find()` in $O(\log n)$ time per operation.

Furthermore, beginning with an empty ScapegoatTree, any sequence of m `add()` and `remove()` operations results in a total of $O(m \log(m))$ time spent during all `rebuild(u)` calls.

Sorted Sets

	find, add, remove
Skip Lists	expected $O(\log n)$
Binary Search Trees	$O(\text{height}) \in O(n)$
Random Binary Trees	expected $O(\log n)$ [find]
Treap	expected $O(\log n)$
Scapegoat Trees	$O(\log n)$ [find] $O(\log n)$ [add*, remove*] amortized $O(\log n)$ [rebuild]

* without a rebuild

Sorted Sets

	find, add, remove
Skip Lists	expected $O(\log n)$
Treap	expected $O(\log n)$
Scapegoat Trees	$O(\log n)$ [find, add*, remove*] amortized $O(\log n)$ [rebuild]

Turns out that in practice the scapegoat tree is often slower than the other SSets (with *expected* $\log n$ runtimes).

It can sometimes spend a lot of time rebuilding.